

C++ 연습 문제: 08. 상속과 다형성

1. 객체지향 설계에서 여러 클래스의 공통된 상태와 기능을 분리하여 계층 구조를 만드는 문법은 무엇인가? (하)

1. 다형성
2. 추상화
3. 캡슐화
4. 상속

2. 파생 클래스(자식 클래스)가 기반 클래스(부모 클래스)의 기능을 물려받을 때, 기반 클래스의 멤버 중 외부 일반 코드에서는 접근할 수 없지만 파생 클래스 내부에서는 접근할 수 있도록 허용하는 지시자는 무엇인가? (하)

1. public
2. private
3. protected
4. virtual

3. 기반 클래스의 포인터나 참조를 통해 파생 클래스의 객체를 다룰 때, 객체의 실제(동적) 타입에 따라 재정의된 함수가 올바르게 호출되도록 하는 C++의 기능은 무엇인가? (하)

1. 가상 함수(Virtual Function)
2. 정적 함수(Static Function)
3. 인라인 함수(Inline Function)
4. 친구 함수(Friend Function)

4. 다음 빈칸에 들어갈 알맞은 낱말은? (중) [클래스 내부에 가상 함수가 하나라도 선언되면, 컴파일러는 실행 시점에 객체의 실제 타입을 판별하고 올바른 함수 주소를 찾아가기 위해 ()을(를) 생성한다.]

1. 가상 함수 테이블(Virtual Table)
2. 힙 메모리(Heap Memory)
3. 제어 블록(Control Block)
4. 참조 카운트(Reference Count)

5. 파생 클래스에서 기반 클래스의 가상 함수를 재정의할 때, 실수로 함수 이름이나 매개변수를 잘못 적어 오버라이딩 대신 새로운 함수가 만들어지는 실수를 컴파일러가 잡아내도록 지정하는 키워드는 무엇인가? (하)

1. final
2. static
3. override
4. explicit

6. 가상 함수를 선언할 때 `virtual void Draw() = 0;`과 같이 작성하여, 이 클래스를 상속받는 파생 클래스가 해당 함수를 반드시 구현하도록 강제하는 함수를 무엇이라고 하는가? (중)

1. 추상 가상 함수
2. 순수 가상 함수
3. 정적 가상 함수
4. 인라인 가상 함수

7. 순수 가상 함수를 하나라도 포함하고 있어, 그 자체로는 불완전하여 객체를 직접 생성(인스턴스화)할 수 없는 클래스를 무엇이라고 하는가? (중)

1. 구체 클래스(Concrete Class)
2. 기반 클래스(Base Class)
3. 추상 클래스(Abstract Class)
4. 봉인 클래스(Sealed Class)

8. 기반 클래스의 포인터로 파생 클래스 동적 객체를 가리키고 있을 때, **delete**를 호출해도 파생 클래스의 소멸자가 정상적으로 호출되게 하려면 반드시 어떻게 설계해야 하는가? (상)

1. 파생 클래스의 소멸자를 **private**으로 선언한다.
2. 파생 클래스의 소멸자를 기반 클래스와 같은 이름으로 통일한다.
3. 기반 클래스의 소멸자를 가상(**virtual**) 소멸자로 선언한다.
4. 기반 클래스의 소멸자에 **= delete**를 추가한다.

9. 다형성을 사용하는 상속 계층에서 기반 클래스의 소멸자를 가상 함수로 선언하지 않았을 때 발생할 수 있는 가장 심각한 문제는 무엇인가? (중)

1. 컴파일 오류가 발생하여 프로그램이 실행되지 않는다.
2. 기반 클래스의 소멸자만 호출되어, 파생 클래스에서 독자적으로 할당한 동적 자원이 해제되지 않아 메모리 누수가 발생한다.
3. 파생 클래스의 소멸자가 중복으로 두 번 호출된다.
4. 프로그램 실행 속도가 크게 저하된다.

10. 파생 클래스에서 기반 클래스와 이름이 같은 함수를 새로 정의하면, 매개변수 구성이 다르더라도 기반 클래스의 오버로딩된 함수들이 가려져 파생 클래스 객체에서 직접 호출할 수 없게 되는 현상을 무엇이라고 하는가? (상)

1. 이름 공간 오염(Namespace Pollution)
2. 오버라이딩(Overriding)
3. 오버로딩(Overloading)
4. 이름 가림(Name Hiding)

11. 다음 빈칸 (A)에 들어갈 알맞은 키워드는? (상) [이름 가림(Name Hiding) 현상을 해결하고, 파생 클래스에서 기반 클래스의 기존 오버로딩 함수들을 함께 호출하여 사용하려면 파생 클래스 내부에 (A) 기반클래스명::함수이름; 을 선언해야 한다.]

1. namespace
2. public
3. using
4. extern

12. 가상 함수에 기본 인수(Default Argument)를 지정했을 때, 다형적 호출(basePtr->Function())에서 발생하는 현상으로 가장 올바른 것은? (상)

1. 기본 인수와 호출되는 함수 모두 정적 타입(컴파일 시점)을 기준으로 결정된다.
2. 기본 인수와 호출되는 함수 모두 동적 타입(실행 시점)을 기준으로 결정된다.
3. 기본 인수는 동적 타입을 따르고, 호출되는 함수는 정적 타입을 따른다.
4. 기본 인수는 정적 타입을 따르고, 호출되는 함수는 동적 타입을 따른다.

13. 다형성을 가진 객체를 복사할 때, 외부에서 객체의 실제 구체 타입을 알지 못하더라도 동일한 타입의 새로운 객체를 동적으로 생성하여 반환하도록 설계하는 가상 함수 패턴(Idiom)은 무엇인가? (중)

1. 싱글턴(Singleton) 패턴
2. 클론(Clone) 패턴
3. 어댑터(Adapter) 패턴
4. 옵저버(Observer) 패턴

14. 실행 시점에 다형성 객체의 실제 타입을 검사하고, 안전하게 파생 클래스 타입의 포인터나 참조로 다운캐스팅(Downcasting)할 때 사용하는 형 변환 연산자는 무엇인가? (중)

1. static_cast
2. reinterpret_cast
3. dynamic_cast
4. const_cast

15. dynamic_cast를 포인터 타입에 적용했을 때, 객체의 실제 타입이 변환하려는 대상 타입과 일치하지 않거나 상속 관계가 아니면 어떤 결과가 발생하는가? (중)

1. std::bad_cast 예외를 던진다.
2. nullptr를 반환한다.
3. 쓰레기값 주소를 반환한다.
4. 컴파일 오류가 발생한다.

16. 객체의 구체적인 타입을 확인하기 위해 dynamic_cast나 RTTI(실행 시간 타입 정보)를 이용한 조건 분기문을 남용할 때 발생하는 구조적 문제점으로 가장 올바른 것은? (상)

1. 가상 함수 테이블의 크기가 기하급수적으로 커진다.
2. 새로운 파생 클래스를 추가할 때마다 캐스팅 분기문 코드를 찾아 모두 수정해야 하므로 확장이 어렵고 객체지향의 의미가 퇴색된다.
3. 프로그램의 메모리 사용량이 극심하게 증가한다.
4. 다이아몬드 상속 구조가 강제로 만들어진다.

17. 파생 클래스의 객체가 생성될 때와 소멸할 때 생성자 및 소멸자가 호출되는 순서로 올바른 것은? (하)

1. 생성 시 파생 클래스부터, 소멸 시 파생 클래스부터
2. 생성 시 기반 클래스부터, 소멸 시 파생 클래스부터
3. 생성 시 파생 클래스부터, 소멸 시 기반 클래스부터
4. 생성 시 기반 클래스부터, 소멸 시 기반 클래스부터

18. 기반 클래스의 생성자 내부에서 가상 함수를 호출했을 때 나타나는 동작으로 올바른 것은? (상)

1. 객체의 동적 타입에 따라 파생 클래스에서 재정의된 가상 함수가 정상적으로 호출된다.
2. 아직 파생 클래스의 멤버가 초기화되기 전이므로, 파생 클래스의 함수가 아닌 기반 클래스에 정의된 버전의 함수가 호출된다.
3. 순수 가상 함수가 호출되어 프로그램이 비정상 종료된다.
4. 컴파일러가 이를 감지하고 빌드를 거부하여 오류를 낸다.

19. 하나의 파생 클래스가 두 개 이상의 기반 클래스를 동시에 상속받는 구조를 무엇이라고 하는가? (하)

1. 단일 상속(Single Inheritance)
2. 가상 상속(Virtual Inheritance)
3. 다중 상속(Multiple Inheritance)
4. 순환 상속(Circular Inheritance)

20. 다이아몬드 상속 구조에서 최상위 기반 클래스의 상태가 중복해서 여러 개 생성되는 모호성 문제를 해결하기 위해, 중간 기반 클래스들이 최상위 클래스를 상속받을 때 사용하는 키워드는 무엇인가? (상)

1. static
2. override
3. virtual
4. protected